

M-03 IPC Module — 16-Sprint Roadmap

Module: M-03 Inter-Process Communication

Layer: 2 (Services)

Owner: KernelArch Labs

Last updated: 2026-06-15

Branch convention: `feature/m03-sprint<N>`;

Overview

M-03 delivers Astra's zero-copy IPC engine. Every sprint adds one security or capability layer on top of the previous one, producing a fully-tested, documented increment that is merged to `main` before the next sprint starts.

The module's core claim (Paper 1 thesis):

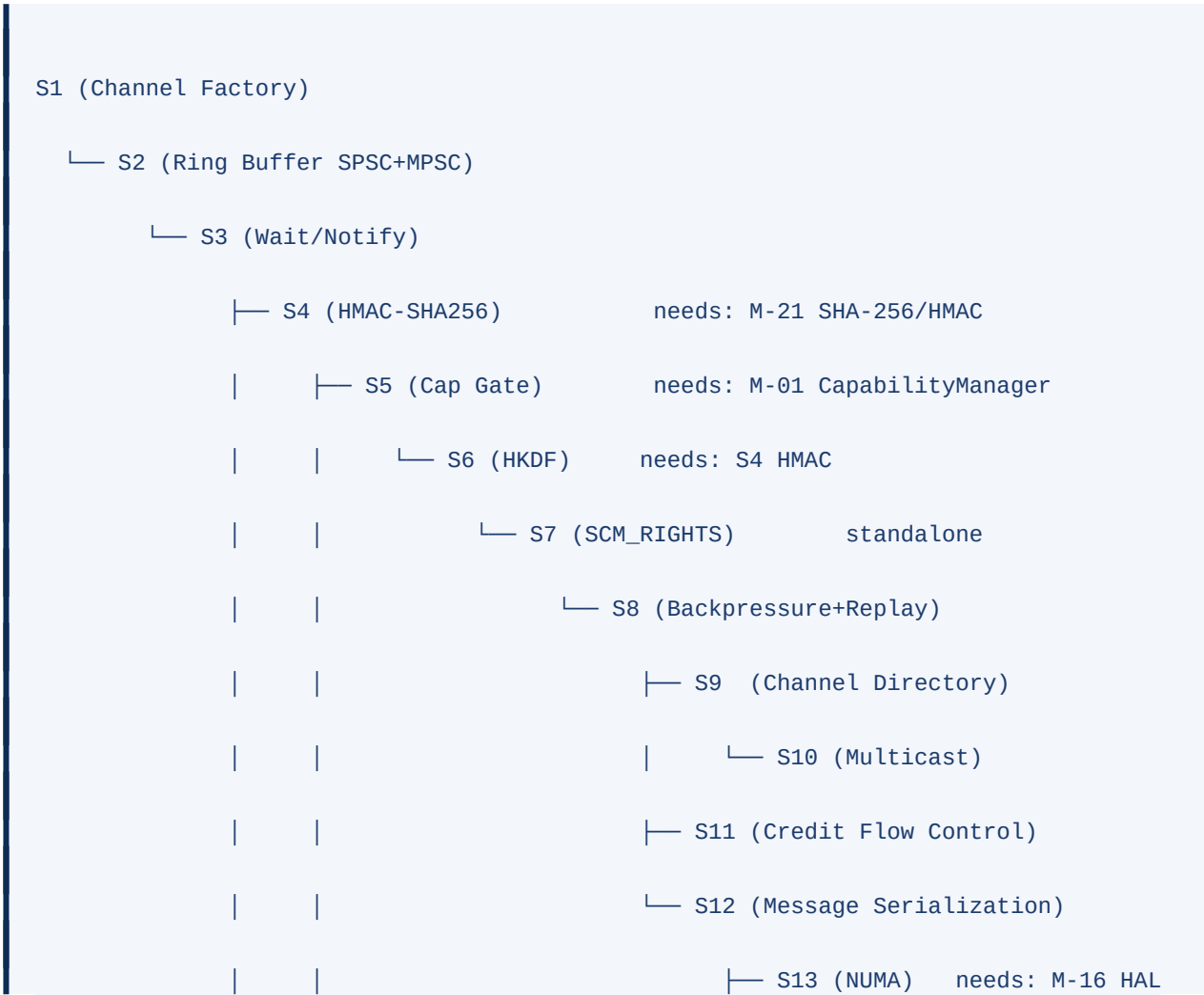
- > Capability-mediated IPC can match raw shared-memory throughput while
 - > supporting global O(1) revocation — without kernel patches.
-

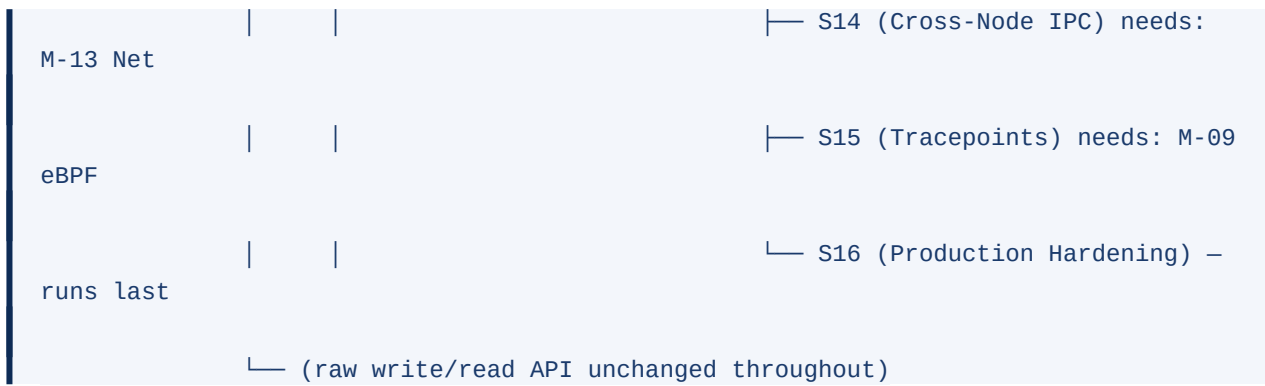
Sprint Status Summary

#	Goal	Status	Branch / Tag
1	Channel Factory (memfd + mmap)	DONE	merged to main
2	Ring Buffer SPSC + MPSC	DONE	merged to main
3	Wait / Notify (futex blocking read)	DONE	merged to main
4	HMAC-SHA256 integrity on every message	DONE	<code>feature/m03-sprint4-5</code>
5	Capability token gate (M-01 integration)	pending	—
6	HKDF session-key derivation per channel	pending	—
7	SCM_RIGHTS memfd passing (cross-process)	pending	—
8	Backpressure + replay guard + seq validation	pending	—

#	Goal	Status	Branch / Tag
9	Channel Directory — named channels, ModuleId lookup, auto-cleanup	pending	—
10	Multicast — one-to-many channels, subscriber groups, fan-out	pending	—
11	Credit-based Flow Control + queue depth monitoring	pending	—
12	Message Serialization — zero-copy framing, schema validation	pending	—
13	NUMA-aware placement (M-16 HAL + mbind)	pending	—
14	Cross-Node IPC — TCP/Unix transport, multi-node routing	pending	—
15	Tracepoints + M-01 control-plane IPC	pending	—
16	Production Hardening — benchmarks, fuzz, stress tests	pending	—

Dependency Graph





Sprint 1 — Channel Factory

Goal: Create a zero-copy IPC channel backed by `memfd_create` + `mmap`.

What was built

- `ChannelFactory::createChannel()` — allocates `memfd`, truncates to `sizeof(ChannelControlBlock) + ring_bytes`, maps `MAP_SHARED`.
- `ChannelFactory::mapExistingChannel()` — maps a peer's fd into the caller's address space (used after `SCM_RIGHTS` in Sprint 7).
- `Channel` RAI wrapper — owns the fd and mapping, moveable.
- `ChannelControlBlock` — exactly 3 cache lines:
 - Line 0 (write side): `write_index` (committed), `write_claim_index` (MPSC)
 - Line 1 (read side): `read_index`
 - Line 2 (meta): channel ID, sizes

Wire format



Key tests

- `Sprint1IpcChannelFactoryTest` — create, inspect control block, map from fd

- `Sprint1IpcExtendedTest` — simulate `SCM_RIGHTS` by forking and passing fd
-

Sprint 2 — Ring Buffer (SPSC + MPSC)

Goal: Framed message delivery with multi-producer safety.

What was built

- `RingBuffer::write()` — two-phase MPSC claim+commit protocol:
- ≤ 256 B payload: wait-free `fetch_add` claim, SKIP sentinel on overcommit
- $\searrow > 256$ B payload: lock-free CAS retry loop
- `RingBuffer::read()` — skips SKIP sentinel records transparently
- `RingBuffer::peekNextSize()` — inspect next payload size without consuming
- Helper predicates: `isEmpty()`, `isFull()`, `freeBytes()`, `usedBytes()`

Wire format per message (Sprints 2–3)

```
[MessageHeader 8 B][Payload N B]

└─ m_uPayloadBytes: N

└─ m_uSequenceNo:  monotone counter (0xFFFFFFFF = SKIP sentinel)
```

Key tests

- `Sprint2IpcRingBufferTest` — framing, FIFO order, wraparound, backpressure
 - `Sprint2IpcMpscTest` — concurrent producers, SKIP sentinel, drain invariant
-

Sprint 3 — Wait / Notify (futex blocking read)

Goal: Reader sleeps on futex instead of busy-polling.

What was built

- `RingBuffer::writeNotify()` — calls `write()` then `atomic::notify_one()`

- `RingBuffer::readWait()` — sleeps via `atomic::wait()` (Linux futex) until a writer calls `notify_one()`; no CPU waste on idle channels

Key tests

- `Sprint3IpcWaitNotifyTest` — producer/consumer across threads; no busy-wait on idle ring

Sprint 4 — HMAC-SHA256 Integrity ✓ DONE

Goal: Every message is authenticated end-to-end. A tampered byte anywhere in the ring is detected before delivery.

Dependencies introduced

- **M-21 ASM Core** — `asm_sha256`, `asm_hmac_sha256_*`
(pure C++ stubs; NASM constant-time port is M-21 Phase 2)

New types

```
struct HmacKey { std::array<U8, 32> m_arrBytes; bool isNull() const; };  
  
struct HmacTag { std::array<U8, 32> m_arrBytes; };
```

New API

```
Result<void> writeHmac      (const HmacKey&, const void*, U32);  
  
Result<U32>  readVerify    (const HmacKey&, void* buf,   U32);  
  
Result<void> writeNotifyHmac (const HmacKey&, const void*, U32);  
  
Result<U32>  readWaitVerify (const HmacKey&, void* buf,   U32);
```

Wire format per authenticated message

```
[MessageHeader 8 B][Payload N B][HmacTag 32 B]
```

```
Total ring bytes = 8 + N + 32
```

HMAC domain input

```
channel_id (U32) || seq_no (U32) || payload_len (U32) || payload (N B)
```

Key tests (7/7 passing)

1. SHA-256 KAT (NIST FIPS 180-4 empty-string vector)
 2. Round trip with valid key
 3. Null key rejected (write + read)
 4. Payload tamper → `HMAC_VERIFICATION_FAIL`
 5. Tag tamper → `HMAC_VERIFICATION_FAIL`
 6. Wrong key → `HMAC_VERIFICATION_FAIL`
 7. `writeNotifyHmac` + `readWaitVerify` blocking pair
-

Sprint 5 — Capability Token Gate (M-01 integration)

Goal: Every IPC operation is gated on a valid `CapabilityToken` with `IPC_SEND` or `IPC_RECV` permission. A revoked token produces `PERMISSION_DENIED` with zero ring footprint.

Dependencies

- **M-01 Core** — `CapabilityManager::validate(token, perm)` (already built)

New types

|

```

struct CapabilityGate {

    astra::core::CapabilityManager* m_pManager;

    astra::core::CapabilityToken    m_token;

    bool isNull() const;

};

```

New API

```

Result<void> writeGated      (const CapabilityGate&, const void*, U32);

Result<U32>  readGated      (const CapabilityGate&, void*, U32);

Result<void> writeNotifyGated (const CapabilityGate&, const void*, U32);

Result<U32>  readWaitGated   (const CapabilityGate&, void*, U32);

```

Validation rules

- `write*Gated` checks `IPC_SEND` before touching the ring claim.
- `read*Gated` checks `IPC_RECV`; re-validates after each futex wake.
- On `PERMISSION_DENIED`: ring is untouched — `write_claim_index` not advanced.

Key tests (7 cases)

1. Bound gate (`IPC_SEND + IPC_RECV`) — round trip succeeds
 2. Unbound gate (`null()`) — `PERMISSION_DENIED` on both sides
 3. Missing `IPC_SEND` — write rejected, ring footprint = 0
 4. Missing `IPC_RECV` — read rejected
 5. Revoke mid-flight — pre-revoke write succeeds, post-revoke fails
 6. Cascading revoke — parent revoke kills child token
 7. Gate + HMAC combined — both checks must pass
-

Sprint 6 — HKDF Session Key Derivation

Goal: Both channel endpoints derive an identical 32-byte session key independently from a shared secret using HKDF-SHA256 (RFC 5869).

Dependencies

- Sprint 4 HMAC (HKDF is built on HMAC-SHA256)

New M-21 API

```
void asm_hkdf_sha256_extract(salt, salt_len, ikm, ikm_len, prk[32]);  
  
void asm_hkdf_sha256_expand(prk[32], info, info_len, out, out_len);
```

Key derivation

```
PRK = HKDF-Extract(salt=channel_id_bytes, IKM=shared_secret)  
  
key = HKDF-Expand(PRK, info="astra-ipc-v1", len=32)
```

Key tests (5 cases)

1. Both endpoints with same inputs → identical 32-byte key
2. Different channel_id → different key (domain separation)
3. Different shared_secret → different key
4. Derived key passes HMAC round-trip (S4 integration)
5. RFC 5869 Test Case 1 KAT

Sprint 7 — SCM_RIGHTS memfd Passing

Goal: Pass a channel's memfd to an unrelated process (not a child) via a Unix domain socket, enabling cross-process IPC without fork.

New API

```
// In ChannelFactory

Result<void>    sendFd(int sock_fd, int channel_fd);

Result<int>     recvFd(int sock_fd);

Result<Channel> mapReceivedChannel(int fd, ChannelId id, SizeT mapped_bytes);
```

Key tests (4 cases)

1. Parent creates channel, sends fd to unrelated process — receiver maps it
 2. Receiver calls `readVerify` — succeeds
 3. Sender closes fd — receiver's mapping stays valid (memfd ref-count)
 4. Double-close protection — second close does not corrupt receiver
-

Sprint 8 — Backpressure + Replay Guard + Sequence Validation

Goal: (a) Slow receivers block fast senders instead of dropping.

(b) Replayed messages are detected and rejected.

Backpressure

- `RingBuffer::writeBlocking(data, len, timeout_ns)` — blocks when ring is full; reader calls `notify_one()` on `read_index` after each consume.

Replay guard

- `m_uSequenceNo` validated on read path against `m_uLastDeliveredSeq`.
- Out-of-order or re-delivered seq → `REPLAY_DETECTED`.

Key tests (4 cases)

1. Ring fills → writer blocks → reader drains → writer unblocks
2. Writer times out (`TIMEOUT`) past deadline

3. Stale seq re-injected → `REPLAY_DETECTED`
 4. Sequence wraparound near `U32_MAX` handled correctly
-

Sprint 9 — Channel Directory ← NEW

Goal: A named channel registry so any module can look up a channel by human-readable name or `ModuleId` without passing raw fd handles.

Automatic cleanup removes stale entries when a process exits.

Design

```
ChannelDirectory  (singleton, lives in M-01 Runtime)

├─ register(name, ModuleId, Channel&) → Result<void>

├─ lookup(name)      → Result<Channel*>

├─ lookup(ModuleId) → Result<Channel*>

├─ unregister(name) → void

└─ cleanup(pid)     → void    (called on PROC_KILL / process exit)
```

- Backed by a flat hash map in a pre-allocated pool (no heap in hot path).
- Thread-safe via a readers-writer spinlock.
- Channel names are null-terminated strings ≤ 64 characters.
- `ModuleId` is the existing `U32` module identifier from M-01.

New header

```
include/astra/ipc/ChannelDirectory.hpp

src/ipc/ChannelDirectory.cpp
```

Key tests (6 cases)

1. Register channel by name → lookup by name returns it
 2. Register channel → lookup by ModuleId returns it
 3. Register duplicate name → `ALREADY_EXISTS`
 4. Unregister → subsequent lookup returns `NOT_FOUND`
 5. `cleanup(pid)` removes all channels owned by that pid
 6. 1000 concurrent lookups from 8 threads — no data race (TSan clean)
-

Sprint 10 — Multicast (fan-out delivery) ← NEW

Goal: One writer sends to a named subscriber group; each subscriber receives an independent copy without any extra kernel copy.

Design

```
MulticastGroup

├─ create(name, max_subscribers) → Result<MulticastGroupId>

├─ subscribe(group_id)           → Result<Channel>   (gives subscriber its own
ring)

├─ unsubscribe(channel_id)       → void

└─ publish(group_id, data, len) → Result<U32>       (count of subscribers
reached)
```

- Each subscriber gets its own dedicated `Channel` with an independent ring buffer — true zero-copy fan-out (the payload is written once per subscriber ring, not copied to a central buffer).
- `publish()` iterates subscriber rings and calls `writeHmac` on each.

A slow subscriber that is full receives `RESOURCE_EXHAUSTED`; the publisher continues to other subscribers (non-blocking fan-out).

- Subscriber list is protected by a hazard-pointer scheme

(no lock in the publish hot path).

New header

```
include/astra/ipc/MulticastGroup.hpp
```

```
src/ipc/MulticastGroup.cpp
```

Key tests (5 cases)

1. 3 subscribers receive identical payload from one publish call
 2. Slow subscriber (full ring) → skipped, others still receive
 3. Unsubscribe mid-publish — no use-after-free (hazard-pointer safe)
 4. 8 concurrent publishers to same group — all subscribers receive all msgs
 5. Multicast + HMAC — each subscriber ring uses the same group key
-

Sprint 11 — Credit-based Flow Control + Queue Depth Monitoring ← NEW

Goal: Writers are granted a finite number of send credits; they must wait until the receiver returns credits (via an explicit ACK ring) before sending more. Queue depth is observable in real-time.

Why credit-based (vs pure backpressure from Sprint 8)

Sprint 8 blocks the writer when the ring is full — reactive.

Sprint 11 prevents the ring from ever filling — proactive. The receiver pre-authorises how many bytes the sender may queue, matching the classic TCP receive-window model.

Design

|

```

struct FlowControlConfig {

    U32 m_uInitialCredits;    // bytes the sender may queue at start

    U32 m_uCreditThreshold;  // receiver returns credits in this-sized chunks
};

class CreditController {

    Result<void> acquireCredits(U32 bytes, U64 timeout_ns);  // sender calls

    void          returnCredits (U32 bytes);                // receiver calls

    U32           queueDepth   () const noexcept;           // observable

    U32           availableCredits() const noexcept;

};

```

- Credits are an atomic counter in the `ChannelControlBlock` meta line.

No extra shared memory allocation.

- `queueDepth() = ring.usedBytes()` — exposed as a named metric for

M-09 eBPF to sample via the tracepoint in Sprint 15.

- The `CreditController` wraps a `RingBuffer` and enforces the credit window before delegating to `writeHmac`.

Key tests (5 cases)

1. Sender uses credits normally — receiver returns them after read
 2. Sender exhausts credits → blocks until receiver returns credits
 3. Credit return from wrong thread — credits still accurate (atomic)
 4. `queueDepth()` tracks ring fill level in real time across threads
 5. Credit starvation timeout — `TIMEOUT` returned after deadline
-

Sprint 12 — Message Serialization ← NEW

Goal: A zero-copy serialization framework for structured messages.

Writers describe a schema; the ring buffer writes a packed binary record.

Readers validate the schema before handing data to the application.

Design

MessageSchema	– field descriptors (type, offset, size, flags)
MessageBuilder	– constructs a packed binary message in a scratch buffer
MessageView	– read-only, zero-copy view over a received payload
SchemaRegistry	– maps U32 schema_id → MessageSchema

Wire layout for serialized messages (extends Sprint 4 format):

```
[MessageHeader 8B][SchemaId U32][FieldCount U16][Pad 2B][Fields ...][HmacTag 32B]
```

Supported field types: U8/U16/U32/U64, I8/I16/I32/I64, F32/F64,

Blob (length-prefixed byte array), String (UTF-8, length-prefixed).

Key tests (6 cases)

1. Build a 4-field message → serialize → deserialize → field values match
 2. Schema mismatch (wrong schema_id on read) → INVALID_ARGUMENT
 3. Truncated payload (write cut short) → PRECONDITION_FAILED
 4. Zero-copy: MessageView.blob() returns pointer into ring memory directly
 5. 1 million round-trips — no heap allocation (checked with ASan)
 6. HMAC covers serialized payload — tamper detected via Sprint 4 path
-

Sprint 13 — NUMA-Aware Placement (M-16 HAL + mbind)

Goal: Channel ring buffers are bound to the NUMA node local to the writer, reducing cross-node memory traffic and p99 latency.

Dependencies

- **M-16 HAL** — minimal `NUMAHal`:
- `cpuToNode(cpu_id)` — which NUMA node owns this CPU
- `bindMemory(ptr, len, node)` — wraps `mbind(MPOL_BIND)`
- `currentNode()` — NUMA node of the calling thread

New API

```
Result<Channel> ChannelFactory::createChannelOnNode(  
    ChannelId id, SizeT ring_bytes, int numa_node = -1  
);
```

Key tests (4 cases)

1. Channel created on node 0 — `mbind` succeeds (skip if single-node system)
2. LLC-miss counter lower for same-node vs cross-node (`perf_event_open`)
3. Auto-detect: bound node matches `currentNode()` of creating thread
4. Non-NUMA fallback: `mbind` returns `ENOSYS` → silently ignored

Sprint 14 — Cross-Node IPC ← NEW

Goal: Two Astra instances on different machines communicate over TCP or Unix domain sockets using the same `RingBuffer` API. A transparent transport layer serializes ring messages and delivers them to the remote ring.

Dependencies

- **M-13 Network** — basic TCP listener/connector (must be at least

Sprint 1 of M-13 before this sprint can start)

- Sprint 12 Message Serialization (cross-node uses the same wire format)

Design

```
IpBridge      - connects two ChannelFactory instances over a transport

├─ createBridge(local_channel, remote_addr) → Result<IpBridgeHandle>

├─ poll()    - drains local ring, serializes, sends over transport

└─ receive() - reads from transport, writes to local ring


Transport (interface)

├─ TcpTransport  - AF_INET SOCK_STREAM

└─ UnixTransport - AF_UNIX SOCK_STREAM
```

- Framing over the transport: `[Length U32][Serialized message]`
- HMAC from Sprint 4 is preserved end-to-end — the tag is transmitted

and verified on the remote side.

- Routing: each bridge is associated with a `ChannelId`; a `MessageRouter` maps `(dst_node_id, channel_id) → IpBridgeHandle`.

Key tests (5 cases)

1. Two processes on loopback (Unix socket) exchange 1000 messages — all arrive
 2. HMAC verified on remote side — tampered transit message rejected
 3. Schema-validated message survives cross-node round-trip (Sprint 12)
 4. Reconnect after transport drop — bridge re-establishes, no message loss
 5. Routing: `MessageRouter` delivers to correct channel by `(node, channel_id)`
-

Sprint 15 — Tracepoints + M-01 Control-Plane IPC

Goal: M-09 eBPF tracepoints fire on every channel lifecycle event.

Channel creation/destruction is commanded via M-01 Runtime's event bus.

Tracepoints (M-09 USDT probes)

```
ipc:channel_created (channel_id, ring_bytes, pid)
ipc:channel_named   (channel_id, name, module_id) ← Sprint 9
ipc:message_sent    (channel_id, seq_no, payload_bytes)
ipc:message_recv    (channel_id, seq_no, payload_bytes)
ipc:hmac_fail       (channel_id, seq_no, pid) ← security event
ipc:cap_denied      (channel_id, perm_required, pid) ← security event
ipc:replay_detected (channel_id, seq_no, expected_seq) ← Sprint 8
ipc:multicast_pub   (group_id, subscriber_count, bytes) ← Sprint 10
ipc:channel_closed  (channel_id, pid)
```

Control-plane IPC (M-01 event bus)

- `Runtime::createIpcChannel(id, ring_bytes, name)` — creates, registers in directory, emits `channel_created` + `channel_named` probes
- `Runtime::revokeIpcChannel(id)` — revokes capability tokens, closes `memfd`, emits `channel_closed` probe

Key tests (5 cases)

1. Create channel via Runtime API → `ipc:channel_created` probe fires
2. Send message → `ipc:message_sent` fires with correct `seq_no`
3. HMAC failure → `ipc:hmac_fail` fires (audit log entry created)
4. Revoke channel → subsequent writes return `CAPABILITY_REVOKED`

5. Full lifecycle: create → send/recv → revoke → correct probe sequence

Sprint 16 — Production Hardening ← NEW

Goal: Prove the full stack is correct under adversarial conditions.

No new features — this sprint only adds benchmarks, fuzz harnesses, and stress tests that cover the entire M-03 surface.

Benchmark suite ([benchmarks/ipc/](#))

Benchmark	What it measures
bench_raw_latency	p50/p99/p99.9 write → read latency, raw SPSC
bench_hmac_overhead	gate cost of HMAC per payload size (64B → 64KB)
bench_cap_gate_overhead	gate cost of capability validate per call
bench_throughput_spSC	msgs/sec + bytes/sec, SPSC, gate ON vs OFF
bench_throughput_mpsc	producer sweep {1,2,4,8,16}, msgs/sec
bench_multicast_fanout	publish latency vs subscriber count {1,4,16,64}
bench_cross_node_rtt	loopback round-trip latency via IpcBridge
bench_pool_scaling	validate() latency vs token pool size 16..4096

All benchmarks emit CSV to [artefact/paper1_*.csv](#) for Paper 1 figures.

Fuzz harnesses ([tests/ipc/fuzz/](#))

Harness	What it fuzzes
fuzz_ring_reader	Feed random bytes as ring content — no crash
fuzz_hmac_verify	Random payloads + tags — always HMAC_VERIFICATION_FAIL or OK
fuzz_schema_deserialize	Random bytes as serialized message — no crash, no UB
fuzz_bridge_recv	Random bytes from transport — no crash, no ring corruption

Stress tests ([tests/ipc/](#))

Test	Scenario
stress_mpsc_24h	8 producers + 1 consumer for 60 s — no data loss, no deadlock
stress_multicast_churn	Subscribers join/leave while publish runs — no use-after-free
stress_credit_starvation	Sender and receiver fight over credits — never deadlocks

Test	Scenario
------	----------

stress_channel_directory 64 threads register/unregister/lookup — TSan clean

Key tests summary

- All benchmarks produce CSV output without crashing
- All fuzz harnesses run 1M iterations under ASan + UBSan — zero findings
- All stress tests run 60 seconds without deadlock, data loss, or memory error

Module Files

include/astra/ipc/			
Types.hpp	– ChannelControlBlock, HmacKey, HmacTag		(S1, S4)
ChannelFactory.hpp	– Channel, ChannelFactory, UniqueFd		(S1, S7)
RingBuffer.hpp	– RingBuffer (grows each sprint)		(S2-S12)
ChannelDirectory.hpp	– named channel registry		(S9)
MulticastGroup.hpp	– fan-out publisher		(S10)
CreditController.hpp	– credit-based flow control		(S11)
MessageSchema.hpp	– schema descriptors, builder, view		(S12)
IpcBridge.hpp	– cross-node transport bridge		(S14)
src/ipc/			
ChannelFactory.cpp			
RingBuffer.cpp			
ChannelDirectory.cpp		(S9)	

MulticastGroup.cpp (S10)

CreditController.cpp (S11)

MessageSchema.cpp (S12)

IpcBridge.cpp (S14)

tests/ipc/

test_sprint1_channel_factory.cpp ✓ DONE

test_sprint1_extended.cpp ✓ DONE

test_sprint2_ring_buffer.cpp ✓ DONE

test_sprint2_mpsc.cpp ✓ DONE

test_sprint3_wait_notify.cpp ✓ DONE

test_sprint4_hmac.cpp ✓ DONE

test_sprint5_cap_gate.cpp pending

test_sprint6_hkdf.cpp pending

test_sprint7_scm_rights.cpp pending

test_sprint8_replay_guard.cpp pending

test_sprint9_channel_directory.cpp pending ← NEW

test_sprint10_multicast.cpp pending ← NEW

test_sprint11_flow_control.cpp pending ← NEW

test_sprint12_serialization.cpp pending ← NEW

test_sprint13_numa.cpp pending

test_sprint14_cross_node.cpp pending ← NEW

test_sprint15_tracepoints.cpp pending

test_sprint16_stress.cpp	pending ← NEW
fuzz/	
fuzz_ring_reader.cpp	pending ← NEW (S16)
fuzz_hmac_verify.cpp	pending ← NEW (S16)
fuzz_schema_deserialize.cpp	pending ← NEW (S16)
fuzz_bridge_recv.cpp	pending ← NEW (S16)
bench/	
bench_raw_latency.cpp	pending ← NEW (S16)
bench_hmac_overhead.cpp	pending ← NEW (S16)
bench_throughput_spesc.cpp	pending ← NEW (S16)
bench_throughput_mpsc.cpp	pending ← NEW (S16)
bench_multicast_fanout.cpp	pending ← NEW (S16)
bench_cross_node_rtt.cpp	pending ← NEW (S16)
bench_pool_scaling.cpp	pending ← NEW (S16)
benchmarks/ipc/	artefact CSVs for Paper 1

Build & Test

```
# Configure (debug build, tests ON)

cmake -S . -B build -DCMAKE_BUILD_TYPE=Debug -DASTRA_ENABLE_TESTS=ON
```

```
# Build everything

cmake --build build --parallel

# Run all IPC tests

ctest --test-dir build -R "Sprint[0-9]+Ipc" --output-on-failure

# Run only a specific sprint

ctest --test-dir build -R Sprint5IpcCapGateTest --output-on-failure

# Run fuzz harness (S16, requires ASTRA_ENABLE_FUZZING=ON)

cmake -S . -B build/fuzz -DASTRA_ENABLE_FUZZING=ON

cmake --build build/fuzz --parallel

./build/fuzz/tests/ipc/fuzz_ring_reader -max_total_time=60
```

Convert this file to PDF

```
# Requires pandoc + texlive-latex-base

pandoc docs/M03_IPC_Sprint_Roadmap.md \

    -o docs/M03_IPC_Sprint_Roadmap.pdf \

    --pdf-engine=pdflatex \

    -V geometry:margin=2cm \

    -V fontsize=11pt \

    --toc
```